

June 8, 2026

Using Agentic Tooling to Build Application-Specific AppSec Controls

A model for turning repeated application security judgment into versioned enforcement artifacts that encode application-specific invariants and produce reviewable evidence.

Proof-Driven AppSec SAST Evidence AI Security Agents

Abstract

General-purpose application security tools are constrained by the assumptions they can safely make across many codebases. They can detect known vulnerability patterns, suspicious API usage, dependency exposure, and common misconfigurations. The local rules that make a particular application safe require a different kind of context. The tool needs to know which resources must be tenant-bound, which serializers are allowed to expose sensitive fields, which state transitions require audit records, which partner webhooks must verify signatures before parsing, and which dependency findings matter because the vulnerable path is actually reachable.

Many of the most important application security failures involve application-specific invariants alongside universal vulnerability classes. These invariants often exist informally in code review memory, engineering convention, wiki pages, framework helpers, and senior developer judgment. They are too local for generic scanners to enforce by default, and historically they have been too expensive to encode as custom tooling for every application.

Agentic tooling changes that cost model. A senior application security engineer, working with an AI-assisted development harness, can now inspect a codebase, identify repeated security-relevant patterns, propose local

invariants, generate enforcement tools, create fixtures, test against known-good and known-bad paths, and integrate the resulting checks into CI or review workflows in days.

This paper argues that application-specific AppSec controls belong between generic scanners and human review. In this model, the agent helps build enforcement artifacts while the security decision remains with the human reviewer. The output is a repository-owned static check, code generation rule, test harness, CI gate, review bot, or analyzer plugin that encodes a narrow invariant and produces reviewable evidence when the invariant is violated. The purpose is to convert repeated AppSec judgment into readable, repeatable, reliable controls while preserving human responsibility for the judgment itself.

1. Introduction

Most AppSec programs use broad tools against failures that are specific to one system.

The usual tools are designed for broad portability, while the application's most important security rules are often specific to its ownership model, data model, workflow, and history.

A scanner can know that raw SQL concatenation is dangerous, that a dependency has a published CVE, that a cookie should be `HttpOnly`, that secrets should stay out of source control, or that a route lacks an obvious authorization decorator. These checks are useful at the level of portable security knowledge.

Real AppSec work often depends on application-specific context beneath those portable checks.

An invoice should be loaded through an organization-scoped lookup. A customer-support export should omit internal notes unless the requester has

a specific role. A webhook handler should verify the signature before parsing the body. A state transition from `draft` to `approved` should require a reviewer distinct from the author. A dependency finding should block release when the vulnerable symbol is reachable under the application's runtime conditions.

These application-specific rules form the security contract of the system.

The difficulty is that this contract is usually implicit. It lives in how experienced engineers review code, how framework helpers are expected to be used, how services are organized, and how incidents have shaped local practice. When the contract is violated, the result may look like IDOR, mass assignment, sensitive data exposure, authorization bypass, audit failure, dependency exposure, or business logic abuse. The root failure is often more precise. The implementation stopped obeying a local invariant that the rest of the application quietly depends on.

General-purpose scanners are poorly positioned to enforce these invariants because they lack permission to assume them. This application may require all invoice reads to pass through `Invoice.for_actor(actor)`.

`AccountPublicSerializer` may be safe while `AccountSerializer` is internal-only. `verify_partner_signature()` may need to dominate parsing for a particular webhook family. The repository has to make those expectations visible before tooling can enforce them.

Historically, custom enforcement of these rules was expensive. A team might write a few Semgrep rules, some CodeQL queries, or a custom script for the most painful recurring issue, but deep application-specific enforcement required expertise, program analysis skill, fixture design, and maintenance time. Most organizations could not justify building bespoke AppSec tooling for every local invariant.

AI-assisted engineering changes the economics by making it practical to use agents as tooling accelerators. A prompt asking an LLM to “review the

application” produces an answer that is difficult to audit and easy to lose. Agentic tooling aimed at local enforcement artifacts produces something the repository can keep, test, and improve.

The agent helps read the codebase, compare sibling paths, identify repeated patterns, draft the invariant, generate the checker, produce tests, run the tool, explain false positives, and iterate. The human AppSec engineer remains responsible for judgment. The repository gets a control it can run again.

The agent session should leave versioned repository tooling behind.

2. Generic Scanners and Local Invariants

Generic scanners have to survive many codebases. That constraint shapes their behavior.

They prefer patterns that are broadly meaningful. They look for dangerous functions, known APIs, dependency metadata, language-specific syntax, framework conventions, and common vulnerability signatures. This makes them valuable as a baseline layer. A security program without general SAST, dependency scanning, secret scanning, and basic configuration checks is leaving obvious coverage on the floor.

But the same universality that makes generic tools deployable also limits what they can know.

A generic scanner may recognize an ORM call and even recognize that a request parameter flows into a query. Correct scoping for this application’s ownership model requires local context. A scanner may detect a serializer that emits many fields. Audience-specific field policy has to be encoded somewhere before the tool can enforce it. A scanner may flag a dependency CVE. Reachability through the application’s real execution paths requires deeper application-specific analysis.

That context is the difference between pattern detection and invariant enforcement. Vulnerability classes travel across applications, while invariants express the local rules a specific system depends on.

Kern's description of Secure by Design at Google is the closest predecessor to this framing. Security posture emerges from the developer ecosystem, and important security properties should be stated as invariants for the system and its tooling to uphold ([Kern, 2024](#)).

For direct object references, the portable vulnerability class warns that authorization may be missing. The local invariant is more precise. Every route that reads or mutates tenant-owned invoices must bind actor, organization, and invoice before the first database effect, and the object used at the effect must be the bound object.

For sensitive data exposure, the portable class warns that serializers can emit fields carelessly. The local invariant names the actual exposure policy. Public account responses may expose email and display name, but never recovery tokens, internal notes, fraud flags, password metadata, or encrypted seed material, regardless of serializer inheritance.

For dependency risk, the portable class says that CVEs should be remediated. The local invariant describes when a finding blocks release. The vulnerable symbol must be reachable, the exploit preconditions must be satisfied, and attacker-controlled input must reach the vulnerable behavior under deployed configuration.

Local invariants are what senior reviewers actually enforce. They are why two code paths that both “have authentication” can receive very different security judgments. They are why a critical CVE can be urgent in one service and documented risk in another. They are why the same serializer pattern can be harmless internally and dangerous on a public route.

The argument of this paper is that agentic tooling makes these local invariants cheap enough to encode.

3. Application-Specific AppSec Controls

An application-specific AppSec control is a repository-owned enforcement mechanism for a local security invariant.

The control might be a static rule, a framework-aware analyzer pass, a Semgrep or CodeQL query, a generated test, a CI gate, a review bot, a code generation constraint, or a repository linter. It might also be a migration checker, a serializer exposure inventory, a dependency reachability note generator, or a domain-specific rule embedded in the build. The mechanism is secondary. The control has to encode a local expectation and produce evidence when the expectation appears to drift.

A good control has four parts.

First, it has an invariant statement narrow enough to test. “Authorization should be good” is too broad to enforce, while “All organization invoice mutations must use an actor-scoped invoice lookup before mutation” gives the tool a concrete relationship to check.

Second, it has positive examples. The control should know what compliant paths look like. These examples may be fixtures, real source spans, known-safe routes, or generated test cases.

Third, it has negative examples. The control should be tested against intentionally violating paths, known historical bugs, or synthesized cases that demonstrate the failure mode.

Fourth, it has a reviewable finding artifact. When the control fires, it should explain the route, actor source, resource lookup, effect, missing binding, comparable safe paths, and remaining uncertainty.

These properties let a team turn repeated AppSec judgment into infrastructure. A valuable first control is narrow, explicit, testable, and reviewable. Perfection matters less than catching one high-value invariant violation with low noise. In practice, that local analyzer may be more useful than a general scanner that produces hundreds of findings no one trusts.

4. Agentic Tooling as a Cost-Curve Shift

The main change is cost. Application-specific security enforcement has always been possible. A strong security engineer could read a codebase, learn its local rules, and write a custom checker. The limiting factor was cost, because custom analysis required time, tooling fluency, language expertise, framework knowledge, test design, and maintenance discipline. Agentic tooling lowers several of those costs at once.

The agent can help map the codebase. It can identify route handlers, serializers, database access patterns, authorization helpers, and repeated naming conventions. It can compare sibling paths and surface repeated structures. It can draft a proposed invariant from examples. It can write the first version of a rule, generate fixtures, run the tool, inspect failures, revise the implementation, and document the result.

The agent becomes useful without becoming authoritative. The human engineer still supplies judgment about which invariant matters, whether the examples are representative, whether the checker is overfitted, whether the false positives are acceptable, and whether the finding artifact proves what it claims. The agent supplies throughput.

The distinction between agentic review and agentic tooling is durability. Agentic review asks a model to inspect code and produce an answer. Useful answers can still be ephemeral, difficult to regression test, and unlikely to improve the repository after the conversation ends.

Agentic tooling asks a model to help create an enforcement artifact. The artifact stays in the repository. It can be tested, reviewed, versioned, tuned, and improved. It can run on every pull request, fail a build, warn a reviewer, generate a decision record, or attach evidence to a security ticket. The agent's most valuable contribution is the machinery that makes future human judgments cheaper.

5. The Enforcement Pipeline

A practical workflow for agentic invariant enforcement has seven stages.

5.1 Observe

Start with what the code already does.

The source may be a recurring review comment, a previous incident, a known bug class, a repeated framework convention, a scanner finding that requires human interpretation, or a senior engineer's intuition that "this class of path should always do X before Y."

At this stage, the agent can help gather examples. It can search for similar routes, find naming patterns, identify helper usage, inspect call sites, and produce a rough map of how the application currently expresses the behavior.

At this stage, the work product is evidence. The useful material is examples, counterexamples, helper names, call paths, and review notes that may later justify a rule.

5.2 State the invariant

The next step is to write the invariant plainly.

A useful invariant should name the subject, the condition, the effect, and the required relationship.

Examples follow.

“Any route that mutates an invoice must load the invoice through an actor- and-organization-scoped query before the mutation.”

“Any public serializer for account data must exclude recovery credentials, fraud flags, internal notes, and encrypted seed material.”

“Any partner webhook handler must verify the request signature before parsing the body or performing any state-changing effect.”

“Any critical dependency finding may be marked non-blocking only when the vulnerable symbol is unreachable or the exploit preconditions are unsatisfied under deployed configuration.”

The invariant should be narrow enough that a tool can be wrong in understandable ways. If the rule is too broad, every violation becomes an argument. If the rule is narrow, violations become reviewable.

5.3 Identify evidence

The agent should collect examples and counterexamples.

Positive evidence includes compliant code paths, standard helpers, established service objects, safe serializers, accepted dependency review notes, or historical fixes.

Negative evidence includes known bugs, synthetic violating fixtures, intentionally unsafe examples, or code paths that lack the expected structure.

The key is to avoid relying on vibes. A local invariant becomes credible when it is supported by the application’s own repeated behavior.

5.4 Generate the control

The control should be as simple as possible.

Sometimes a Semgrep rule is enough. Sometimes a CodeQL query is appropriate. Sometimes the control should be a custom AST walker. Sometimes it should be a framework-aware analyzer that understands routes, middleware, serializers, and ORM calls. Sometimes it should be a generated test that asserts all routes in a family use the approved helper. Sometimes it should be a codegen constraint that makes the unsafe path difficult to express.

The agent can draft multiple versions. The human should choose the one with the best maintenance profile.

The design question should focus on the smallest enforcement artifact that catches the repeated failure with acceptable noise.

5.5 Test against fixtures

A control without fixtures is just another opinion.

Every application-specific AppSec control should include known-good and known-bad cases. Representative fixtures matter more than large fixtures.

A tenant-boundary rule might include one safe route using the actor-scoped helper, one unsafe route using direct lookup, one route that loads through an intermediate service, and one intentionally unresolved dynamic case.

A serializer exposure rule might include one public serializer, one internal serializer, one inherited serializer, and one serializer that exposes a banned field through a nested object.

A dependency reachability control might include one reachable vulnerable symbol, one present-but-unreachable dependency, one configuration-gated path, and one unknown case that must be routed to the unresolved state.

Fixtures are what make the control repeatable. They let the team change the rule without forgetting what it was supposed to protect.

5.6 Produce reviewable findings

A finding should carry enough proof for a reviewer to understand and challenge it without reconstructing the entire path from scratch.

For an authorization invariant, the finding should identify the route, externally controlled parameters, actor source, resource lookup, tenant or ownership binding, dangerous effect, and missing or weaker guard.

For a serializer invariant, it should identify the route or caller, serializer, exposed field, field classification, audience, and policy expectation.

For a dependency invariant, it should identify the package, vulnerable symbol, reachable call path if known, exploit preconditions, runtime configuration, and reason the finding is blocking, warning, accepted, or unresolved.

The control should hand reviewers the smallest artifact that makes the issue inspectable, including the local facts that explain why this path matters in this application.

5.7 Integrate with fail, warn, and accept modes

Not every control should fail every build.

Some invariant violations are release blockers. Some are warnings. Some require security review. Some are accepted risks with expiration dates. Unknowns should route to a human review state.

A useful enforcement tool needs at least four states.

Fail means the invariant violation is clear and the risk is unacceptable for merge or release.

Warn means the path is suspicious, but the evidence is incomplete or the impact is limited.

Accept means the risk is documented, time-bounded, and approved.

Unresolved means the tool lacks enough evidence to prove compliance or violation, and the uncertainty itself is the output.

Block-only AppSec tooling eventually gets bypassed, while tooling that distinguishes failure, warning, acceptance, and uncertainty can fit into normal engineering workflows.

6. Example Invariant Families

6.1 Tenant boundary invariants

Tenant boundary failures are among the clearest examples of application-specific security risk.

A generic scanner can look for direct object references. A local control can enforce the application's actual ownership model.

In one application, tenant binding may require `organization_id` to match a route parameter and a session claim. In another, it may require membership in a workspace. In another, it may require account hierarchy, reseller delegation, regional partition, or project-level permission. The invariant is local.

A useful tenant-boundary control asks whether actor, tenant, and resource are bound before the first sensitive effect. Authentication establishes an actor. Authorization still has to connect that actor to the resource.

A finding might say the following.

The route accepts `invoice_id` from the request. The handler loads `Invoice.find(invoice_id)` directly. The current actor is available as `request.user`. Comparable invoice mutation routes use `Invoice.for_actor(request.user).for_org(org_id).find(invoice_id)`.

This path reaches `invoice.update(...)` without evidence that the invoice belongs to the actor's organization.

That is the difference between a generic IDOR warning and an application-specific invariant finding.

6.2 Serializer exposure invariants

Serializer exposure is another local invariant family.

Serializer exposure enforcement starts with an application decision about sensitive fields, permitted audiences, and serializer roles. The relevant roles may be public, internal, administrative, or partner-facing.

An application-specific control can maintain a field exposure policy and compare serializers against it. It can detect direct exposure, inherited exposure, nested exposure, and accidental inclusion through broad model-shape serialization.

The strongest version of this control models audience and transformation. A field may be safe for internal admin views and unsafe for customer-facing APIs. A field may be safe after transformation and unsafe in raw form. A field may be safe when redacted and unsafe when serialized directly.

The finding artifact should show the serializer, the field, the audience, the route or caller, and the violated exposure expectation.

6.3 Webhook verification invariants

Webhook handlers often have a strict local rule. Verify authenticity before trusting the body.

A generic scanner has limited visibility into the difference between parsing, validation, signature verification, idempotency checks, and state-changing

effects in a given framework. An application-specific control can encode the expected sequence.

For a partner webhook family, the invariant might read as follows.

Verify signature before parsing trusted fields. Validate event type before dispatch. Enforce idempotency before mutation. Log rejection without persisting attacker-controlled data.

The control can inspect the handler's call order, known verification helpers, parsing functions, dispatch paths, and mutation sinks. It can flag handlers that parse before verification, mutate before idempotency, or silently accept unknown event types.

The control should enforce the application's accepted webhook sequence.

6.4 State transition invariants

Many security bugs are invalid state transitions.

A reimbursement needs approval before moving from `submitted` to `paid`. A deployment needs review before moving from `draft` to `production`. A user needs an existing owner or break-glass path before becoming an organization owner. A document needs private attachments removed before becoming public.

These rules belong to the product more than to any universal vulnerability taxonomy.

A local control can model transition families and required preconditions. It can compare peer paths, identify missing review checks, detect audit omissions, and flag direct state assignments that bypass service-layer transitions.

The finding should name the state transition, the effect, the expected preconditions, and the path that bypasses them.

6.5 Dependency reachability invariants

Dependency scanning produces too much raw material and not enough judgment.

A critical CVE in a container image becomes a critical application risk through reachability and exploitability. It may be present but unreachable, require a configuration the application does not use, or be mitigated by sandboxing, runtime flags, or disabled features. It may also be fully reachable and urgent.

The local invariant should require evidence-backed downgrades.

A dependency finding can be downgraded only when reachability, exploit preconditions, runtime configuration, and compensating controls are documented with evidence.

Agentic tooling can help here by generating reachability notes, scanning imports and call paths, checking container usage, reading configuration, and producing a structured decision artifact.

The resulting control encodes the team's dependency triage discipline in the repository so that reachability, exploitability, configuration, and compensating controls are reviewed consistently.

7. Intent Markers and No-Op Annotations

Application-specific controls become easier when developers can mark intent.

An intent marker is a runtime-neutral annotation, decorator, attribute, structured comment, or metadata entry that states what security invariant a path is supposed to satisfy.

The marker makes a security claim machine-readable without changing whether the implementation is actually secure.

For example:

```
@tenant_bound(resource="invoice", tenant="organization")
@audit_required(effect="billing.mutation")
def update_invoice(request, organization_id, invoice_id):
    ...
```

The annotation may be a no-op at runtime. The enforcement tool reads it and asks whether the implementation satisfies the claim.

Where is the actor loaded? Where is the tenant loaded? Where is the invoice loaded? Is the invoice lookup scoped to the actor and tenant? Does the object that was checked reach the mutation? Is an audit record written before or after the irreversible effect? Are unresolved dynamic calls recorded?

The intent marker is an assertion that the enforcement tool must verify against the implementation.

It lets developers declare what kind of review a path requires. It gives tools a stable surface to inspect. It reduces guesswork. It creates a bridge between human design intent and static enforcement. It can be adopted incrementally because no-op markers do not change runtime behavior.

It also creates accountability. A handler marked `tenant_bound` that does not actually bind tenant and resource violates a stated local contract, which is more precise than a generic missing-best-practice warning.

The broader principle is that application-specific security controls should support both inferred invariants and explicit developer-declared invariants, with verification applied to both.

8. Proof Artifacts

Application-specific controls must produce evidence because their authority comes from local reasoning. If a generic scanner flags `eval`, the reviewer already understands the broad danger. If a local invariant tool flags a tenant-boundary violation, the reviewer needs to know why the tool believes the path violates this application's ownership model. Without that proof, the finding becomes backlog noise.

A proof artifact should be small enough to review and complete enough to challenge.

For a route-to-effect invariant, the artifact should name the route or endpoint, request-controlled values, actor source, resource lookup, tenant or ownership binding, dangerous effect, missing or weaker guard, comparable safe paths when available, and source spans for every claim.

For a serializer exposure invariant, the artifact should name the route or caller, serializer or response builder, field, field classification, audience, exposure policy, and the path by which the field reaches the response.

For a dependency reachability invariant, the artifact should name the package and version, vulnerable symbol or behavior, known exploit preconditions, reachable application call paths if any, configuration evidence, compensating controls, and final triage state.

Proof artifacts make agentic tooling usable in security workflows. The agent can help produce them, and each claim must remain inspectable through evidence.

9. Failure Modes

Application-specific enforcement can fail badly.

The first failure mode is overfitting. A tool may encode today's structure too tightly and reject legitimate future changes. This is especially likely when an agent learns from too few examples. The fix is to keep invariants narrow, include fixtures, and record unresolved cases honestly.

The second failure mode is false authority. A generated control may look official because it runs in CI, even if its model is weak. The fix is to require proof artifacts and human review before treating a control as release-blocking.

The third failure mode is noisy enforcement. If a local control produces too many weak findings, developers will route around it. The fix is to support fail, warn, accept, and unresolved states so different signal strengths receive different treatment.

The fourth failure mode is hidden uncertainty. Dynamic dispatch, reflection, metaprogramming, framework magic, and unresolved imports can all hide relevant behavior. A good tool should emit unresolved evidence for unknown paths.

The fifth failure mode is policy drift. An invariant may be correct when written and wrong six months later, so local controls need owners who can update them as the application and risk model change.

The sixth failure mode is agent dependency. A generated tool has to be understandable and maintainable after the original agent session. The generated code must be readable, tested, documented, and versioned like any other security-critical artifact.

10. Why This Beats Generic Tools on Local Risk

Application-specific controls occupy a different layer from generic scanners.

A generic scanner is better at broad baseline coverage. It can detect common dangerous APIs, known CVEs, secrets, insecure configuration, and framework anti-patterns across many repositories. It should remain part of the security stack.

A local invariant tool is better at enforcing the rules that only make sense inside one application. A narrow local control can outperform a powerful general scanner on the invariant it encodes because it is allowed to know local helpers, field classifications, route structure, authorization models, serializer audiences, and risk acceptance rules.

The point is not to replace scanners. It is to add the layer they cannot provide. A mature program uses generic tools for portable risk and application-specific controls for local drift. Generic tools create the baseline. Application-specific controls encode the lessons learned from incidents, reviews, false positives, architecture decisions, and repeated human judgment.

High-value security automation often preserves what the team already learned the hard way and applies it consistently in future changes.

11. SecureSentry as a Verification Case

SecureSentry is the prototype that made this model concrete. It is a Sentry-aware static analyzer built to test whether application-specific invariants could be modeled against a large real application rather than a toy benchmark. Its output posture is explicit. Findings are source-backed review artifacts for human validation.

The analyzer encodes narrow proof policies over Sentry-specific concepts. These include organization, project, and team boundaries, membership lifecycle, durable credential authority, external identity cleanup, service hook secrets, external storage cleanup, cross-service context export, internal proxy request steering, impersonation-sensitive mutation, and secondary-channel disclosure.

This maps directly onto the enforcement pattern in this paper. A recurring review concern becomes a spec. The spec defines the local invariant, the semantic model, implementation slices, acceptance tests, and risk controls. The analyzer extracts facts such as ownership models, lifecycle markers, query shapes, object uses, boundary provenance, and proof gaps. The resulting review artifact names the missing proof and points to source-backed evidence.

Several invariant packs illustrate the difference between generic scanning and local enforcement. The `OrganizationMember` live-invariant pack checks whether a role-bearing membership row is approved and live before it is treated as authority. The `ServiceHook` secret contract pack checks whether creator-known signing material survives creator removal without rotation, invalidation, or a documented owner-independent contract. The bulk lifecycle pack looks for bulk deletion of owner rows where per-instance cleanup may be the only proven external-storage cleanup path. The cross-service context export pack checks whether exported organization context has matching membership and authority proof and treats internal service boundaries as claims that need supporting evidence.

The findings are therefore phrased as TP-shaped review candidates. For example, the `EventAttachment` finding identifies a row deletion path, a V1 blob cleanup path, and the proof needed to downgrade the concern. The `Seer` context finding identifies the membership lookup, exported organization/user context, and the missing approved/live membership proof. That format demonstrates the core claim of this paper in concrete form. Reviewable artifacts should make local security reasoning inspectable without asking the reviewer to trust an agent's final judgment.

SecureSentry also shows how an invariant enforcement app matures. Early signals should be treated as candidates for review. The team runs them against code, commit history, fixtures, and real review examples, measures false positives, and refines the invariant until the finding shape is stable. Once

a check has produced little to no false positives for long enough to earn trust, it can graduate from signal to lintable error. After that, it belongs in CI/CD, where the repository rejects regressions automatically and removes the need to rediscover the same local rule during every review.

The graduation path depends on trust built over repeated runs. A good control should be run across many commits, especially older commits where the team already understands the expected risk shape. If the tool repeatedly finds the same meaningful proof gap without spurious noise, it becomes executable memory for the application's security model.

SecureSentry also shows why agentic tooling matters. Building this kind of analyzer requires repeated movement from code reading to invariant writing to fact extraction to fixture design to finding review. Agentic tooling lowers the cost of that loop. The endpoint is a tested repository control that remembers a security judgment and produces evidence when the code drifts away from it.

12. Implementation Blueprint

A team can start small.

The first application-specific AppSec control should be narrow, painful, and repeated. It should come from a class of issues the team has already seen more than once.

Good first candidates include tenant-scoped resource lookup, public serializer field exposure, webhook verification order, dependency reachability documentation, audit logging before irreversible effects, admin-only state transitions, unsafe direct use of request-controlled identifiers, or repeated suppressions from a generic scanner.

The first implementation should prioritize proving the loop over elegance.

Pick one invariant. Gather five safe examples and two unsafe examples. Ask the agent to summarize the repeated structure. Write the invariant in plain language. Generate a first checker. Add fixtures. Run it. Inspect false positives. Simplify. Add a proof artifact. Decide whether the control should fail, warn, or only report. Put it in CI or review.

Start with one narrowly scoped control that catches one real class of repeated local security failure.

Once that works, the pattern compounds. Each new invariant becomes easier because the repository now has a place for local controls, fixtures, documentation, and review decisions.

Over time, the application accumulates a repository-local enforcement layer that reflects its own security model.

13. Relationship to Shift-Left Security

Shift-left security often fails because it pushes responsibility left without moving enough judgment, context, or tooling left with it.

Developers are told to own security while the security team remains the only group with enough context to make hard calls. This creates frustration on both sides. Developers receive scanner output they cannot interpret, security teams receive pull requests too late to influence architecture, and the decision machinery remains centralized even when everyone agrees security should happen earlier.

Kern argues that developer ecosystems should take responsibility for ensuring key security properties. Application-specific controls are a repository-scale version of that idea. They move local security expectations into tools, fixtures, gates, and review artifacts that developers can actually use while building the system ([Kern, 2024](#)).

Application-specific controls make shift-left more concrete by giving developers narrow, inspectable security work products supported by local tooling.

Instead of asking every developer to become broadly security-fluent, the program can assign narrow security lanes and support them with tools. One developer can own serializer exposure for a service. Another can own dependency reachability notes. Another can own tenant-boundary fixtures. Another can own converting repeated AppSec findings into local checks.

The senior AppSec engineer still owns standards, review quality, escalation, and final judgment. The work product changes. The security team reviews artifacts and spends less time rediscovering every issue from scratch.

This is where agentic tooling and Human Swarm Mode meet. AI compresses the learning loop for each lane, developers become useful narrow security operators, local controls turn their lane knowledge into enforcement, and expert review keeps the system honest.

Distributed AppSec ownership becomes credible when it is grounded in artifacts.

14. Conclusion

Application security needs both better generic scanners and cheaper custom enforcement.

AI-assisted engineering makes it practical to build application-specific controls for the invariants that actually protect a system. Those invariants include tenant boundaries, actor-resource binding, serializer exposure, webhook verification, state transition rules, dependency reachability, audit requirements, and release-gate policy.

The agent acts as a toolsmith, helping turn observed patterns and repeated review judgment into versioned enforcement artifacts.

The human security engineer still decides what matters, while the repository gains machinery that can preserve and reapply those decisions.

A mature AppSec program will use general-purpose tools for portable risk and local invariant tools for application-specific drift. Developers can work in narrow lanes with AI-assisted learning, local enforcement artifacts, and expert review.

A security program that does this stops re-litigating the same review judgment in every pull request. It captures the judgment, tests it, and runs it.

References

Autor, D. H. (2024). *Applying AI to Rebuild Middle Class Jobs*. NBER Working Paper No. 32140. National Bureau of Economic Research. <https://doi.org/10.3386/w32140>

Becker, G. S., & Murphy, K. M. (1992). *The Division of Labor, Coordination Costs, and Knowledge*. The Quarterly Journal of Economics, 107(4), 1137–1160. <https://doi.org/10.2307/2118383>

Braz, L., & Bacchelli, A. (2022). *Software Security during Modern Code Review: The Developer's Perspective*. arXiv:2208.04261. <https://arxiv.org/abs/2208.04261>

Brynjolfsson, E., Li, D., & Raymond, L. R. (2025). *Generative AI at Work*. The Quarterly Journal of Economics, 140(2), 889–942. <https://doi.org/10.1093/qje/qjae044>

Dell'Acqua, F., McFowland III, E., Mollick, E. R., Lifshitz-Assaf, H., Kellogg, K. C., Rajendran, S., Kraye, L., Candelon, F., & Lakhani, K. R. (2023). *Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of Artificial Intelligence on Knowledge Worker Productivity and Quality*. Harvard

Business School Working Paper No. 24-013. <https://www.hbs.edu/faculty/Pages/item.aspx?num=64700>

Kern, C. (2024). *Secure by Design at Google*. Google Security Engineering Whitepaper. <https://research.google/pubs/secure-by-design-at-google/>

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. arXiv:2302.06590. <https://arxiv.org/abs/2302.06590>

Rajapakse, R. N., Zahedi, M., Babar, M. A., & Shen, H. (2021). *Challenges and Solutions when Adopting DevSecOps: A Systematic Review*. arXiv:2103.08266. <https://arxiv.org/abs/2103.08266>

Rajapakse, R. N., Zahedi, M., & Babar, M. A. (2022). *Collaborative Application Security Testing for DevSecOps: An Empirical Analysis of Challenges, Best Practices and Tool Support*. arXiv:2211.06953. <https://arxiv.org/abs/2211.06953>